

SENG492 – SENIOR PROJECT II

Low-Level Design, Spring2026



AttendAI

Project Team Members:

Ahmet Arda Bağdatlı (SENG), System Design
Vugar Gurbanov (SENG)
Nuray Aliyeva (SENG)

Supervisor:

Tansel Dökeroğlu

Jury Members:

Ulaş Güleç
Emin Kuğu

Coordinator:

Dr. Özlem ALBAYRAK

Table of Contents

Related Documents	2
1- INTRODUCTION and SCOPE.....	3
1.1- Object-Design Trade-offs	3
1.2- Interface Documentation Guidelines	3
1.3- UML Diagrams	3
2- PACKAGES.....	4
2.1 Server Package	4
2.2 Client Package	4
3- CLASS INTERFACES	5
3.1- App Navigator	5
3.1.1- Attributes	5
3.1.2- Methods	5
3.1.3 Flask Route Interface.....	6
3.2- Database Manager	7
3.2.1 Attributes.....	7
3.2.2 Methods	7
3.3- Embed Manager	9
3.3.1- Attributes	9
3.3.2- Methods	9
3.4- Webcam Manager	10
3.4.1- Attributes	10
3.4.2- Methods	10
3.5- Session Cache	11
3.5.1- Attributes	11
3.5.2- Methods	11
3.6- Session Config	11
3.6.1- Attributes	11
3.7- Session.....	12
3.7.1- Attributes	12
3.8- Snapshot.....	12
3.8.1- Attributes	12
3.9- Snap Data	13
3.9.1- Attributes	13

3.10- User	13
3.10.1- Attributes	13
3.11- Face Embedding	13
3.11.1- Attributes	13
3.12- Lecture.....	14
3.12.1- Attributes	14
3.12.2- Methods	14
4- GLOSSARY	15
5- REFERENCES	16

Related Documents

Name	URL
High Level Design	https://teduterm-attendai.github.io/AttendAI-Page/documents/AttendAI_HLD.pdf
Requirements	https://teduterm-attendai.github.io/AttendAI-Page/documents/AttendAI_Analysis.pdf

Additional Note: The tabular contents for methods and attributes in the document were formulated with the help of Claude. The design of the project is refined exclusively by team members alone from the previously given [high level design](#) document.

1- INTRODUCTION and SCOPE

This document details the low-level design of AttendAI, an automated attendance management system for TED University using snapshots and a cache system. It covers class interfaces, package structure, Flask route definitions, and system architecture decisions.

1.1- Object-Design Trade-offs

- We have chosen Redis over repeated PostgreSQL queries for embedding lookups during active sessions, at the cost of minor RAM usage.
- All calculations for embedding and comparison will be done on the serverside for accessibility. This is to make the client app small.
- Embeddings are stored in persistent storage with AES-256 encryption. This is to comply with KVKK law.
- Snapshot images and login images are deleted after embedding generation. This is to comply with KVKK law.
- Embedding calculations will be done on the servers to keep images within a our system rather than using cloud computing.

1.2- Interface Documentation Guidelines

All of our system interfaces are documented as Python class method signatures with full parameter types and return types. Flask HTTP routes are documented as a separate route table with method, auth requirements, and role constraints.

1.3- UML Diagrams

We have UML diagrams on our previous [high level design](#) document. This document contains a system, class and a navigation diagram. Their credibility stays mostly the same as we havent changed much from our previous iteration of AttendAI.

2- PACKAGES

We will have a client and server style split for our packages. Since our environment for development will be on two docker containers we can show how the class files are going to be arranged as such.

2.1 Server Package

The server package contains business logic python files, data storage models and the PostgreSQL and Redis storage.

```
server/
├── app_navigator.py
├── database_manager.py
├── embed_manager.py
├── models/
│   ├── user.py
│   ├── session.py
│   ├── session_cache.py
│   ├── session_config.py
│   ├── snapshot.py
│   ├── snap_entry.py
│   ├── face_embedding.py
│   └── lecture.py
```

2.2 Client Package

The client package will house the HTML templates while also having a minor amount of JavaScript to handle webcam logic.

```
client/
├── templates/
│   ├── login.html
│   ├── register.html
│   ├── student/
│   │   ├── lectures.html
│   │   └── sessions.html
│   ├── teacher/
│   │   ├── sessions.html
│   │   ├── session_view.html ← contains JS snapshot logic
│   │   ├── attendance.html
│   │   └── lectures.html
│   └── admin/
│       ├── main.html
│       ├── users.html
│       └── sql.html
```

3- CLASS INTERFACES

3.1- App Navigator

The app navigator is the central Flask application. Registers routes, serves HTML pages, handles HTTP request/responses and holds references to all the other manager instances.

3.1.1- Attributes

Attribute	Type	Description
app	Flask	Flask application instance
activeSession	Session	Current session attached to the instance
dbManager	DatabaseManager	Reference to the database manager
embedManager	EmbedManager	Reference to the embedding manager
webcamManager	WebcamManager	Reference to the webcam manager

3.1.2- Methods

Method	Parameters	Return	Description
__init__	db_url: str, img_path: str	None	Initializes Flask app, creates instances of DatabaseManager, EmbedManager and WebcamManager. Configures JWT cookie settings.
register_routes	—	None	Registers all Flask route handlers for the application
start	host: str, port: int	None	Starts the Flask server
shutdown	—	None	Gracefully closes DB connections and clears active caches
login	username: str, password: str	Response	Verifies credentials via DatabaseManager, generates JWT token, sets HTTP-only cookie, redirects to role-appropriate dashboard
logout	—	Response	Clears JWT cookie, redirects to login page
run_session_loop	—	None	Controls snapshot loop using activeSession. Clears cache and resets activeSession to None in finally block

3.1.3 Flask Route Interface

All of the routes marked with a Yes on Auth are protected by JWT decorators. Roles are enforced per route.

Route	Method	Auth	Role	Description
/login	GET, POST	No	Any	Serves login page, handles login form submission
/register	GET, POST	No	Any	Serves register page. On POST validates uploaded photo has exactly one face before creating user and storing encrypted embedding
/logout	POST	Yes	Any	Clears JWT cookie, redirects to login
/student/lectures	GET	Yes	Student	Lists lectures the student is enrolled in
/student/sessions	GET	Yes	Student	Lists sessions for a given lecture. Includes mailto correction button pre-filled with teacher email, course code, session date and student info
/teacher/sessions	GET	Yes	Teacher	Lists all sessions for the teacher
/teacher/session/<id>	GET	Yes	Teacher	Views a specific session. This fetches all snapshots and snapdata for display
/teacher/session/<id>/edit	POST	Yes	Teacher	Saves edited attendance entries. Also calls update_snap_data per changed SnapEntry
/teacher/session/start	POST	Yes	Teacher	Creates session, preloads cache, starts run_session_loop
/teacher/session/attendance	POST	Yes	Teacher	Receives base64 snapshot from browser, passes to WebcamManager
/teacher/lectures	GET	Yes	Teacher	Lists teacher's lectures
/teacher/lectures/create	GET, POST	Yes	Teacher	Creates a new lecture
/admin	GET	Yes	Admin	Admin main dashboard
/admin/users	GET	Yes	Admin	Lists all system users
/admin/sql	GET, POST	Yes	Admin	SQL executor interface

3.2- Database Manager

Handles all persistent data operations. Coordinates between PostgreSQL for storage and Redis for temporary session cache. All embeddings are encrypted with AES-256 before PostgreSQL storage.

3.2.1 Attributes

Attribute	Type	Description
db	Object	PostgreSQL connection pool

3.2.2 Methods

Method	Parameters	Return	Description
create_user	user: User	User	Hashes password with Argon2id, INSERT into Users, returns created User
get_user	user_id: int	User	SELECT from Users by userID
update_user	user: User	None	UPDATE existing row in Users
set_embedding	embedding: FaceEmbedding	None	AES-256 encrypts embedding, INSERT into FaceEmbeddings
get_embedding	user_id: int	FaceEmbedding	SELECT from FaceEmbeddings by userID, decrypts before returning
create_lecture	lecture: Lecture	Lecture	INSERT into Lectures, returns created Lecture
get_lecture	lecture_id: int	Lecture	SELECT from Lectures by lectureID
add_lecture_student	lecture_id: int, student_id: int	None	INSERT into LecturesStudents, links student to lecture
remove_lecture_student	lecture_id: int, student_id: int	None	DELETE from LecturesStudents. Removing a student from a lecture
get_lecture_students	lecture_id: int	list[User]	SELECT all enrolled students for a lecture from LecturesStudents
create_session	session: Session	Session	INSERT into Sessions, returns created Session
get_session	session_id: int	Session	SELECT from Sessions by sessionID
update_session	session: Session	None	UPDATE existing row in Sessions

Method	Parameters	Return	Description
create_snapshot	snapshot: Snapshot	Snapshot	INSERT into Snapshots + bulk INSERT all snapData SnapEntries into SnapDatas
get_snapshots	session_id: int	list[Snapshot]	SELECT all Snapshots for a session, includes SnapDatas per snapshot
update_snap_data	snap_entry: SnapEntry,	None	UPDATE a single row in SnapDatas , used when teacher edits attendance

3.3- Embed Manager

Handles all face embedding operations using InsightFace buffalo_l model. Generates embeddings from snapshots, compares them against SessionCache and DB student list to determine attendance per snapshot.

3.3.1- Attributes

Attribute	Type	Description
model	InsightFace	Loaded InsightFace buffalo_l model instance

3.3.2- Methods

Method	Parameters	Return	Description
load_model	model_name: str	None	Loads InsightFace model weights into memory on startup
generate_embeddings	image: bytes	list[FaceEmbedding]	Detects all faces in snapshot, returns one FaceEmbedding per detected face
register_embedding	image: bytes	FaceEmbedding	Registration only. Rejects if 0 or 2+ faces detected. Returns single FaceEmbedding if exactly one face found.
compare_embeddings	embed: FaceEmbedding, cache: SessionCache, students: list[User], config: SessionConfig	int None	Checks cache first, falls back to students list. Returns matched studentID or None
_compare_against_cache	embed: FaceEmbedding, cache: SessionCache, threshold: float	int None	Private. Compares against Redis cache. Returns studentID if match above threshold, else None
_compare_against_db	embed: FaceEmbedding, students: list[User], threshold: float	int None	Private. Compares against fetched DB students. Returns studentID if match above threshold, else None
process_snapshot	image: bytes, cache: SessionCache, config: SessionConfig, students: list[User]	list[SnapEntry]	Full pipeline. This generates embeddings, compares each, fills absent students, returns complete SnapEntry list

3.4- Webcam Manager

Handles receiving and temporarily storing snapshot images sent from the client browser. Lives on the server side. Webcam capture is handled client-side via minimal JavaScript `getUserMedia()`. Images deleted immediately after processing for KVKK compliance.

3.4.1- Attributes

Attribute	Type	Description
imgPath	str	Temporary file path to save received image before processing

3.4.2- Methods

Method	Parameters	Return	Description
take_image	image_data: str	bytes	Receives base64 encoded image from browser POST, decodes to bytes, saves temporarily to imgPath
delete_image	—	None	Deletes temporary image file after EmbedManager finishes processing — KVKK compliance

3.5- Session Cache

Temporary Redis-backed store of decrypted embeddings for one active session. Preloaded with enrolled student embeddings on session start. Grows dynamically as unknown faces are identified via DB fallback. Dies when session ends.

3.5.1- Attributes

Attribute	Type	Description
sessionID	int	ID of the session this cache belongs to
redis	Redis	Redis connection instance

3.5.2- Methods

Method	Parameters	Return	Description
add_embedding	user_id: int, embedding: FaceEmbedding	None	Serializes and stores decrypted embedding in Redis. Called on session preload AND when DB fallback finds a new match mid-session.
get_cached_embeddings	user_id: int	FaceEmbedding	Retrieves and deserializes embedding from Redis for a specific user. Returns None if not found.
clear	—	None	Clears the Redis cache. This is called on session end

3.6- Session Config

Data container for instructor-configured session settings. Held by the Session object. Also persisted to PostgreSQL with the Sessions.config column. No methods — purely a data container.

3.6.1- Attributes

Attribute	Type	Description
snapshotInterval	float	Seconds between each snapshot capture
cacheThreshold	float	Minimum confidence score for a cache comparison match
databaseThreshold	float	Minimum confidence score for a DB fallback comparison match

3.7- Session

Represents an active or completed attendance session. Holds both persistent data (pushed to PostgreSQL) and runtime data (cache — lives in Redis only during the session). No methods as this is only to store information about the session for AppNavigator.

3.7.1- Attributes

Attribute	Type	Description
sessionID	int	Primary key. This can be empty for new sessions
lectureID	int	Foreign key referencing Lectures table
snapshots	list[Snapshot]	Builds up during session loop. Each snapshot appended after processing.
config	SessionConfig	Session configuration also persisted to PostgreSQL Sessions.config column
cache	SessionCache	Runtime only Redis-backed embedding store. Preloaded on session start, grows during session. None when session is not active. Dies when session ends.

3.8- Snapshot

A snapshot consists of snap data collected during the session loop. Each snapshot is generated from one image and contains the snapdata of the students.

3.8.1- Attributes

Attribute	Type	Description
snapshotID	int	Primary key can be empty for a new snapshot.
sessionID	int	Foreign key referencing Sessions table
snapshotDate	str	Timestamp of when the snapshot was taken
snapData	list[SnapEntry]	Attendance results for this snapshot. Contains one per enrolled student. Populated by EmbedManager.process_snapshot().

3.9- Snap Data

A single attendance data for a student. Its marked as a boolean that dictates if the student attended in that moment. Also contains a confidence metric.

3.9.1- Attributes

Attribute	Type	Description
snapshotID	int	Foreign key referencing Snapshots table
userID	int	Foreign key referencing Users table
attending	bool	True if student was identified above confidence threshold, False otherwise
confidence	float	Cosine similarity score from embedding comparison

3.10- User

Represents a system user. Email stored for attendance correction mail redirect. No methods because DatabaseManager handles all persistence.

3.10.1- Attributes

Attribute	Type	Description
userID	int	Primary key. This is auto-generated
username	str	Username used for login and display
password_hash	str	Argon2id hashed password. This is never stored plain
email	str	University email address. Stored only for revision messaging
role	enum	STUDENT TEACHER ADMIN

3.11- Face Embedding

Represents a face embedding generated by InsightFace. No methods. Acts as a data container.

3.11.1- Attributes

Attribute	Type	Description
embedID	int	Primary key. Auto-generated on database push.
userID	int	Foreign key referencing Users table
embedding	array	512-float numpy array from InsightFace buffalo_l. Stored AES-256 encrypted in PostgreSQL to comply with KVKK.
model	str	Model name that generated the embedding e.g. 'buffalo_l'

3.12- Lecture

A lecture created by the teacher that stores the participating students.

3.12.1- Attributes

Attribute	Type	Description
lectureID	int	Primary key — auto-generated
teacherID	int	Foreign key referencing Users table (teacher)
students	array	List of enrolled student user IDs
lectureCode	str	Course code identifier e.g. 'SENG-491'
createDate	str	Date the lecture was created

3.12.2- Methods

Method	Parameters	Return	Description
add_student	user_id: int	None	Adds a student ID to the students list for this lecture

4- GLOSSARY

Term	Definition
JWT	JSON Web Token. Stateless token used for session authentication
KVKK	Turkish personal data protection law (Kişisel Verileri Koruma Kanunu)
AES-256	Symmetric encryption standard used for embedding storage in PostgreSQL
Argon2id	Password hashing algorithm. All user passwords hashed before storage
Embedding	512-float numpy array representing a detected face
SessionCache	Temporary Redis store. Created on session start, grows mid-session, cleared on end
SnapEntry	Attendance result for one student from one snapshot (attending bool + confidence)
buffalo_l	InsightFace model used for face embedding generation
Confidence	Cosine similarity score between two embeddings
Redis	In-memory key-value store. This is used for session-scoped embedding cache
PostgreSQL	Relational database. Used for all persistent data
InsightFace	Open-source face detection and embedding library (MIT license)
Docker	Containerisation platform. 2 containers present in the development environment for AttendAI: client and server
flask-jwt-extended	Flask library providing JWT authentication decorators

5- REFERENCES

- 1- Hynek, S. (n.d.). *argon2-cffi: Argon2 for Python*. Read the Docs. <https://argon2-cffi.readthedocs.io/>
- 2- Kişisel Verileri Koruma Kurumu. (n.d.). *KVKK resmî sitesi*. <https://www.kvkk.gov.tr/>
- 3- Pallets Projects. (n.d.). *Flask: A Python micro-framework*. <https://flask.palletsprojects.com/>
- 4- PostgreSQL Global Development Group. (n.d.). *PostgreSQL: The world's most advanced open-source relational database*. <https://www.postgresql.org/>
- 5- Python Software Foundation. (n.d.). *Python*. <https://www.python.org/>
- 6- pyca. (n.d.). *Cryptography: Cryptographic recipes and primitives for Python*. <https://cryptography.io/>
- 7- Redis Ltd. (n.d.). *Redis: The real-time data platform*. <https://redis.io/>
- 8- Serengil, S., & Ozpinar, A. (n.d.). *InsightFace: Face recognition & analysis toolkit*. <https://www.insightface.ai/>
- 9- Voit, M. (n.d.). *flask-jwt-extended documentation*. Read the Docs. <https://flask-jwt-extended.readthedocs.io/>